

Learning Rust in 30 Days — By Building a Game

For a developer coming from Java / Spring Boot.

The whole plan is built around one growing project: a text adventure (`tiny_dungeon`) that evolves into a small game engine. You learn each Rust concept the moment the project *needs* it — never as abstract theory. Java/Spring analogies are called out throughout so you can lean on what you already know.

Daily rhythm: ~60-90 minutes. Read a little, build a lot, and always end the day with code that compiles and runs.

The one rule: When the borrow checker rejects your code, don't just fix it — read the error out loud and ask *why* it's complaining. In Rust, reading compiler errors **is** the skill.

Which plan should I follow?

- **Have only 3 days / want a taste?** Do the **3-Day Sprint** (Days 1-3 below). You'll finish a complete, playable game with save/load. You'll be "productive but shallow" — fine for a demo, not yet fluent.
 - **Want to actually learn Rust?** Do the full **30 days**. The first 3 days are the sprint; the rest is where fluency is built.
-

Week 1 — Foundations (this is also your 3-Day Sprint, extended)

Goal: go from "Java with weird syntax" to understanding ownership, and finish a playable game.

Day 1 — Core loop, structs, enums

- **Concepts:** `let` vs `let mut` (variables are immutable by default — opposite of Java), `struct` ($\approx$ a Java class with only fields), `enum` (Rust enums are *far* stronger than Java's — they can hold data, like sealed classes / records with a fixed set of shapes), `match`.
- **Build:** Turn your hardcoded rooms into real data. Create a `Room` struct and a `Player` struct. Represent directions with an enum.
- **Java analogy:** `enum Direction { North, South, East, West }` looks familiar, but you'll soon give variants data, which Java can't do cleanly.

Day 2 — Ownership & borrowing (the big one)

- **Concepts:** ownership, moves, borrowing (`&` and `&mut`), why `String` moves but `i32` copies. This is the single most important day.
- **Build:** Store rooms in a `HashMap<String, Room>`. Move the player between rooms. You *will* hit "borrow checker" errors here — that's the point.
- **Java analogy:** In Java every object is a shared reference the GC cleans up. Rust makes you say who owns a value and who's just borrowing it. Slow down here.

Day 3 — Error handling + save/load (Sprint finish line)

- **Concepts:** `Option<T>` ($\approx$ `Optional`), but you *can't* ignore it), `Result<T, E>` ($\approx$ checked exceptions, but returned as a value), the `?` operator, and the `serde` + `serde_json` crates.
- **Build:** Add an inventory (`Vec<Item>`), a `look`/`take`/`inventory` command set, and save/load the game to a JSON file.
- **Cargo note:** `cargo add serde --features derive` and `cargo add serde_json`. `Cargo.toml` is your `pom.xml`; crates.io is Maven Central.
-  **End of Day 3 = complete playable game. This is the whole 3-Day Sprint.**

Day 4 — Modules & project structure

- **Concepts:** `mod`, `pub`, splitting code across files, the module tree.
- **Build:** Break the single `main.rs` into `player.rs`, `world.rs`, `commands.rs`.
- **Java analogy:** Like packages and `public`/`private`, but visibility is private by default and controlled per-item.

Day 5 — Iterators & closures

- **Concepts:** `.iter()`, `.map()`, `.filter()`, `.collect()`, closures (`|x| x + 1`).
- **Build:** Rewrite loops (e.g. listing inventory, finding items) using iterator chains.
- **Java analogy:** Almost identical to Java Streams and lambdas — this will feel like home.

Days 6-7 — Consolidate

- Re-read the ownership chapter of *the book* now that you've felt the pain. It'll click much harder.
 - Refactor the game. Add 2-3 rooms and a puzzle (e.g. a locked door needing a key from your inventory).
-

Week 2 — Traits, generics, and idiomatic Rust

Goal: replace Java's inheritance instincts with Rust's trait-based thinking.

Day 8 — Traits (Rust's interfaces)

- **Concepts:** defining and implementing traits, default methods.
- **Build:** A `Describable` trait implemented by `Room`, `Item`, and `Player`.
- **Java analogy:** Traits $\approx$ interfaces with default methods. But Rust has **no inheritance** — you compose behavior with traits instead of extending classes. Unlearn `extends`.

Day 9 — Derive macros & common traits

- **Concepts:** `#[derive(Debug, Clone, PartialEq)]`, the `Display` trait, `Default`.
- **Build:** Make your structs printable and comparable. Use `{:?}` debug formatting everywhere.

Day 10 — Generics

- **Concepts:** generic functions and structs, trait bounds (`fn foo<T: Display>(x: T)`).
- **Java analogy:** Like Java generics but with *no type erasure* — real monomorphized code, and bounds are enforced hard at compile time.

Day 11 — Collections deep-dive

- **Concepts:** `Vec`, `HashMap`, `HashSet`, `String` vs `&str` (the classic beginner confusion).
- **Build:** A proper world map with connections between rooms stored as data.

Day 12 — Enums with data + exhaustive matching

- **Build:** Model game events/commands as an enum (`Command::Go(Direction)`, `Command::Take(String)`, `Command::Quit`) and `match` on it. Parse user text into this enum.
- **This is where Rust enums genuinely beat Java.**

Days 13-14 — Consolidate

- Add a simple combat system (an `Enemy` struct, health, attack) using traits and enums.
 - Write your first **unit tests** (`#[test]`, `cargo test`) — like JUnit but built into the language.
-

Week 3 — The hard middle: lifetimes, smart pointers, real error handling

Goal: face the concepts that make people bounce off Rust — with a project that needs them.

Day 15 — Lifetimes

- **Concepts:** why lifetimes exist, `<'a>` syntax, when the compiler needs your help.
- **Reality check:** This feels alien coming from GC languages. Go slow. You rarely write lifetimes by hand day-to-day, but you must be able to *read* them.

Day 16 — `Box`, and the heap

- **Concepts:** `Box<T>` for heap allocation and recursive types.
- **Build:** A dialogue tree or nested room description that needs indirection.

Day 17 — `Rc` and `RefCell` (shared/mutable state)

- **Concepts:** `Rc<T>` (shared ownership), `RefCell<T>` (interior mutability), the combo `Rc<RefCell<T>>`.
- **Java analogy:** This is the closest Rust gets to "just a shared mutable object like Java." Understand *why* it's the exception, not the default.

Day 18 — Idiomatic error handling

- **Concepts:** custom error enums, the `?` operator across error types, the `anyhow` and `thiserror` crates.
- **Build:** Replace every `.unwrap()`/`.expect()` in your game with proper error propagation.

Day 19 — CLI polish with `clap`

- **Build:** Add real command-line flags (`--load savefile.json`, `--new`) using the `clap` crate.
- **Spring analogy:** `clap`'s derive API feels like annotation-driven config — declarative arg parsing.

Days 20-21 — Consolidate

- Full refactor pass. Everything modular, tested, no panics on bad input.
 - Add persistent high scores or a save-slot system.
-

Week 4 — Bringing it together (+ a Spring-flavored bonus)

Goal: ship something you're proud of, and see where Rust fits in your existing world.

Day 22 — Testing & documentation

- **Concepts:** integration tests, doc comments (`///`), doc tests, `cargo doc`.
- **Java analogy:** Like Javadoc, but your code examples in docs are actually *run as tests*.

Day 23 — Traits as generics vs `dyn` (dynamic dispatch)

- **Concepts:** `impl Trait`, `Box<dyn Trait>`, static vs dynamic dispatch.
- **Build:** A list of heterogeneous game entities behind a common trait.

Day 24 — Concurrency basics

- **Concepts:** threads, `Arc`, `Mutex`, message passing with channels.
- **The payoff:** Rust's "fearless concurrency" — the borrow checker prevents data races at *compile time*. This is the moment the pain pays off.

Days 25-26 — BONUS: a tiny web API with `axum`

- **Why:** This is your Spring Boot bridge. Build a small REST endpoint that exposes your game (e.g. `GET /rooms`, `POST /command`).
- **Spring analogy:** `axum` routing $\approx$ Spring controllers; `serde` $\approx$ Jackson; async/`tokio` $\approx$ the reactive stack. You'll feel at home fast — this is where a Java dev often goes "oh, I could actually use this at work."
- ⚠ Async is genuinely hard. It's a *bonus*, not a requirement. Don't let it demoralize you if it's fuzzy.

Days 27-28 — Capstone build

- Pick one: (a) polish the game into something genuinely fun with 10+ rooms, combat, and a win condition, **or** (b) turn it into a web-served game with the `axum` API. Make it a repo you'd show someone.

Days 29-30 — Ship & reflect

- Write a `README`, add tests, run `cargo clippy` (Rust's linter — treat its advice as a free senior-dev code review) and `cargo fmt`.
- Push to GitHub. Write down the 5 things that confused you most; those notes are gold for the next project.

Beyond day 30

Rust competence is built project by project. Good next steps once the game is done: a real CLI tool you'd actually use, a small web service in `axum` (your Spring instincts transfer well), or contributing a small fix to an open-source Rust crate.

The mindset shifts to expect (Java → Rust)

- **Immutable by default.** You add `mut` when you *want* mutation, not the reverse.
- **No `null`.** `Option<T>` forces you to handle absence. This eliminates a whole class of bugs.
- **No exceptions.** Errors are values (`Result`) you must handle or explicitly propagate with `?`.
- **No inheritance.** Compose with traits; you'll stop reaching for `extends`.
- **No garbage collector.** You think about ownership instead. This is the tax — and the reason Rust is fast and safe at once.
- **The compiler is a pair-programmer, not a gate.** Its errors are unusually good. Read them fully.

Best free resources

- *The Rust Programming Language* ("the book") — the official, excellent tutorial.
- *Rust by Example* — runnable snippets for when you want to see a concept fast.
- *Rustlings* — small compiler-driven exercises; great for the ownership weeks.
- `cargo clippy` — install and run it constantly; it teaches idioms as you go.